

Problem Statement: Computational Performance Characterization of Neural ODEs

Final Year Project (FYP) — Track 2: Computational Characterization

September 2026

1 Problem Statement

Neural Ordinary Differential Equations (Neural ODEs) represent continuous-time dynamical processes by parameterizing the time derivative of a state using a neural network vector field,

$$\frac{dy(t)}{dt} = f_{\theta}(y(t), t), \quad y(t_1) = y(t_0) + \int_{t_0}^{t_1} f_{\theta}(y(t), t) dt, \quad (1)$$

where the continuous trajectory is evaluated numerically via an adaptive or fixed-step ODE solver. In machine learning literature, the computational cost of a Neural ODE is commonly summarized using the *Number of Function Evaluations (NFE)*—the count of times the neural network f_{θ} is invoked during numerical integration.

However, an aggregate NFE count does not directly represent the actual wall-clock time and hardware resources consumed by each evaluation or solver step. A single function evaluation varies in latency depending on network depth and width, while the numerical solver itself incurs step-adaptation, interpolation, and tableau overheads. Furthermore, under stiff dynamics, implicit solvers introduce substantial linear-algebra operations (such as Jacobian evaluations and LU decompositions) that NFE counting entirely overlooks.

This project investigates how closely NFE corresponds to actual wall-clock runtime and practical computational cost, and systematically identifies under what conditions model, solver, numerical, and hardware factors create deviations between NFE and observed computational effort. The primary research question is formulated neutrally:

To what extent does NFE characterize the computational cost of Neural ODEs, and under what conditions do model, solver, numerical, and hardware factors cause deviations between NFE and actual computational cost?

The objective of this work is rigorous computational performance characterization, not the development of a new Neural ODE architecture.

2 Motivation

Reporting NFE alone as a universal proxy for computational performance can obscure critical engineering trade-offs. Two distinct Neural ODE configurations may record similar NFE values yet exhibit substantially different wall-clock runtimes and hardware resource demands because:

- **Vector-Field Complexity:** Each function evaluation has different computational complexity (T_f) depending on model width and depth.
- **Solver Overhead:** Different numerical solvers execute varying amounts of internal housekeeping (error estimation, Butcher tableau multiplications, and rejected steps).
- **Stiffness Dynamics:** Stiff problems force explicit solvers into tiny step sizes or divergence, whereas implicit solvers maintain stability at the expense of non-trivial Jacobian and linear-solver workloads.
- **Numerical Tolerances:** Tighter error tolerances increase integration step counts and change the proportion of accepted versus rejected solver attempts.
- **Batching and State Dimension:** High-dimensional states or mini-batching can introduce within-batch step-synchronization bottlenecks on parallel accelerators.
- **Hardware Execution:** CPU versus GPU execution, kernel launch overheads, and compilation frameworks materially influence actual throughput.

Therefore, reporting NFE alone may not provide a complete picture of practical computational performance. The project investigates when NFE is a faithful proxy and when additional computational factors explain observed deviations.

3 Research Hypothesis

The formal hypothesis to be empirically tested in this study is:

Hypothesis: *NFE is a useful proxy for Neural ODE computational cost, but its relationship with actual wall-clock runtime systematically varies with model complexity, solver characteristics, stiffness, numerical settings, and hardware/implementation.*

This hypothesis is treated as a testable scientific proposition. The experimental outcomes may confirm that NFE tracks runtime tightly under specific regimes, demonstrate that it is only partially representative, or show that solver and hardware factors systematically break the correspondence under identifiable conditions.

4 Literature Basis — 15 Papers in Three Functional Groups

The preliminary literature review identified 15 relevant studies, structured into three functional groups according to their methodological similarity:

- **Group A — MLP-based / Conventional Architectures (6 papers):** Chen et al. (2018), Finlay et al. (2020), Ghosh et al. (2020), Kim et al. (2021), Onken & Ruthotto (2020), and Caldana & Hesthaven (2025).
Why Group A: These works use conventional/MLP-based Neural ODE formulations or closely related formulations, providing the baseline for understanding standard Neural ODE computational structures.

- **Group B — Similar / Modified Architectures (3 papers):** Dupont et al. (2019), Xia et al. (2021), and Poli et al. (2020).
Why Group B: These papers modify the Neural ODE formulation or solver structure (e.g., state augmentation, momentum, hypersolvers) and are useful for understanding how architectural/numerical modifications change computational workload.
- **Group C — Computational Methodology (6 papers):** Rackauckas et al. (2019), Lienen & Günnemann (2022), Kidger (2022, PhD thesis), Zhuang et al. (2020), Zhuang et al. (2021), and Rackauckas et al. (2020).
Why Group C: These works provide computational methodology related to solver implementation, runtime, memory, batching, differentiation, and framework-level efficiency.

5 Top-5 Literature Supporting the Investigation

From the 15-paper corpus, five foundational papers were prioritized based on their direct relevance to computational cost profiling, solver mechanics, and stiffness. Table 1 summarizes their roles.

Table 1: Key Literature Supporting the Computational Characterization

Paper & Authors	Core Focus	Why Selected for This Investigation
Kim et al. (2021) [1]	Stiff Neural ODEs	Directly studies computational behaviour on stiff systems and compares explicit and implicit solvers; provides the main foundation for investigating how stiffness changes solver workload and computational cost.
Finlay et al. (2020) [2]	Regularization & NFE	Directly connects properties of learned Neural ODE dynamics with NFE and wall-clock computational cost; particularly useful for understanding the relationship between learned dynamics and numerical integration effort.
Lienen & Günnemann (2022) [3]	Parallel Solvers & Profiling	Provides detailed solver statistics and implementation-level computational measurements such as loop time and NFE; highly relevant for studying computational overhead beyond a simple function-evaluation count.
Caldana & Hesthaven (2025) [4]	Stiff Systems & Solvers	Directly connects stiffness, solver behaviour, and computational efficiency, making it useful for studying how computational cost changes across stiff regimes.
Dupont et al. (2019) [5]	Augmented Neural ODEs	Provides a useful perspective on how state dimensionality and modified Neural ODE representations can affect solver behaviour and computational workload.

6 Research Gap

Existing literature provides substantial evidence about NFE, solver behaviour, stiffness, runtime, memory, and implementation efficiency. However, these factors are often studied separately or in the context of improving a particular method. Our project focuses specifically on characterizing the relationship between NFE and actual computational cost under controlled changes in model complexity, solver choice, stiffness, numerical settings, and hardware/implementation. The goal is to determine where NFE tracks actual cost well and where additional computational factors explain deviations. The study therefore seeks to identify measurable conditions under which NFE remains a reliable proxy and conditions under which other computational factors explain the observed deviation.

7 Planned Experimental Direction

The experimental investigation will proceed through three controlled stages:

1. **Stage 1 — Basic Controlled Experiment:** Change one computational factor at a time while keeping other conditions fixed, and measure NFE together with actual wall-clock runtime and relevant computational metrics.
2. **Stage 2 — Stiffness/Solver Investigation:** Evaluate Neural ODE behaviour across different dynamical regimes and solver choices, including explicit and implicit approaches where appropriate.
3. **Stage 3 — Scaled Experiment:** Repeat the strongest/basic finding across additional model sizes, stiffness levels, systems, or computational configurations to determine whether the observed relationship is reproducible.

Possible outcomes include: NFE strongly predicts runtime under the tested conditions; NFE predicts runtime only under certain conditions; or additional factors systematically explain significant deviations. The experiments will determine the final conclusion.

8 Scope

This project does not propose a new Neural ODE architecture. It uses an established Neural ODE architecture and studies its computational behaviour under controlled experimental conditions.

References

- [1] S. Kim, W. Ji, S. Deng, Y. Ma, and C. Rackauckas. Stiff neural ordinary differential equations. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 31(9):093122, 2021.
- [2] C. Finlay, J.-H. Jacobsen, L. Nurbekyan, and A. M. Oberman. How to train your neural ODE: the world of Jacobian and kinetic regularization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 3154–3164, 2020.
- [3] M. Lienen and S. Günnemann. torchode: A parallel ODE solver for PyTorch. In *NeurIPS 2022 Workshop on the Symbiosis of Deep Learning and Differential Equations*, 2022.
- [4] M. Caldana and J. S. Hesthaven. Neural ordinary differential equations for model order reduction of stiff systems. *International Journal for Numerical Methods in Engineering*, 2025 (originally arXiv:2408.06073, 2024).
- [5] E. Dupont, A. Doucet, and Y. W. Teh. Augmented neural ODEs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.